```
import 'dart:async';
import 'package:flutter/material.dart';
import 'package:firebase_database/firebase_database.dart';
import 'package:google_maps_flutter/google_maps_flutter.dart';

class BusMapScreen extends StatefulWidget {
  final String busId;

  const BusMapScreen({Key? key, required this.busId}) : super(key:
key);

  @override
  State<BusMapScreen> createState() => _BusMapScreenState();
}

class _BusMapScreenState extends State<BusMapScreen> {
  // Google Map Controller & Marker Set
  GoogleMapController? _mapController;
  final Set<Marker> _markers = {};

  // Firebase Realtime Database reference and listener subscription
  late DatabaseReference _busLocationRef;
  StreamSubscription<DatabaseEvent>? _busSubscription;

  // Initial camera position
  static const CameraPosition _initialCameraPosition = CameraPosition(
    target: LatLng(12.9716, 77.5946), // Default initial coordinates
    zoom: 15.0,
  );

  @override
  void initState() {
    super.initState();
    _listenToBusLocation();
  }

  /// Listens to real-time bus location updates from Firebase Realtime
Database
  void _listenToBusLocation() {
    _busLocationRef =
FirebaseDatabase.instance.ref('buses/${widget.busId}/location');

    _busSubscription = _busLocationRef.onValue.listen((DatabaseEvent
event) {
      final data = event.snapshot.value as Map<dynamic, dynamic>?;

      if (data != null && data['latitude'] != null &&
data['longitude'] != null) {
```

```
        final double latitude = (data['latitude'] as num).toDouble();
        final double longitude = (data['longitude'] as
num).toDouble();
        final LatLng busPosition = LatLng(latitude, longitude);

        // Update the marker on the map
        _updateBusMarker(busPosition);

        // Animate the camera to keep the bus centered
        _mapController?.animateCamera(
          CameraUpdate.newLatLng(busPosition),
        );
      }
    }, onError: (error) {
      debugPrint('Error listening to bus location: $error');
    });
  }

  /// Updates or creates the bus marker with updated latitude and
longitude
  void _updateBusMarker(LatLng position) {
    setState(() {
      _markers.removeWhere((marker) => marker.markerId == const
MarkerId('bus_marker'));

      _markers.add(
        Marker(
          markerId: const MarkerId('bus_marker'),
          position: position,
          icon:
BitmapDescriptor.defaultMarkerWithHue(BitmapDescriptor.hueViolet),
          infoWindow: InfoWindow(
            title: 'Bus #${widget.busId}',
            snippet: 'Lat: ${position.latitude.toStringAsFixed(4)},
Lng: ${position.longitude.toStringAsFixed(4)}',
          ),
        ),
      );
    });
  }

  @override
  void dispose() {
    // Cancel the Firebase stream subscription when the widget is
disposed
    _busSubscription?.cancel();
    _mapController?.dispose();
    super.dispose();
```

```
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: GoogleMap(
        initialCameraPosition: _initialCameraPosition,
        markers: _markers,
        onMapCreated: (GoogleMapController controller) {
          _mapController = controller;
        },
      ),
    );
  }
}
```